

# APUNTE INTEGRAL DE CÁTEDRA: INGENIERÍA DE SOFTWARE Y GESTIÓN DE CONFIGURACIÓN (SCM)

## 1. INGENIERÍA DE SOFTWARE

### 1.1 LA NATURALEZA DEL SOFTWARE Y LA CRISIS DE LA DISCIPLINA

La evolución histórica del software ha transitado desde los esfuerzos artesanales de las primeras décadas hasta la imperiosa necesidad de una rigurosidad científica e industrial. Comprender la naturaleza abstracta del software constituye la piedra angular para cualquier ingeniero; a diferencia del hardware, el software representa conocimiento formalizado. No se "agota" por el uso físico, sino que se deteriora bajo la presión del cambio constante y la entropía de su estructura. El fracaso en proyectos de gran escala rara vez se origina en la carencia de herramientas, sino en una incomprensión profunda de las estructuras conceptuales que estamos construyendo: sistemas de una complejidad sin precedentes que desafían la intuición humana.

#### **Definición Formal y la Pirámide de la Representación**

Para esta cátedra, el software trasciende la mera escritura de código. Se define como un conjunto integral compuesto por **programas, procedimientos, reglas, documentación y datos asociados**. Siguiendo la perspectiva de Pantaleo & Rinaudo (2018), lo conceptualizamos como "conocimiento representado en distintos niveles de abstracción".

Esta **pirámide de la representación** permite visualizar el flujo del desarrollo desde la abstracción máxima hasta la ejecución física:

1. **Nivel de Requerimientos Conceptuales:** El concepto de negocio, la arquitectura lógica y el diseño conceptual (Máxima abstracción).

2. **Especificación de Diseño:** Diagramas de arquitectura, modelos de datos, reglas de negocio y planes de ejecución.
3. **Implementación (Código Fuente):** Realización en lenguajes de programación (e.g., Python, Java, C++).
4. **Software de Base y Plataformas de Ejecución:** El sistema operativo, bibliotecas de enlace y frameworks de soporte.
5. **Lógica Binaria:** Código binario y microinstrucciones (El secreto de la máquina).
6. **Hardware Físico:** Impulsos eléctricos en el silicio y memoria física (Mínima abstracción/Máximo nivel físico).

## Análisis de la "Crisis del Software"

Históricamente, la disciplina ha enfrentado problemas sistémicos recurrentes que definen este estado de crisis:

- **Insatisfacción del cliente:** El producto final no guarda correspondencia con las necesidades reales del negocio.
- **Inflexibilidad:** Dificultad extrema para extender o adaptar el sistema, convirtiendo el mantenimiento en una tarea de alto riesgo.
- **Documentación deficiente:** Ausencia de registros técnicos que permitan la trazabilidad y el traspaso de conocimiento.
- **Baja fiabilidad:** Presencia de defectos críticos en ambientes de producción debido a procesos de verificación inexistentes.
- **Desbordes presupuestarios:** Tiempos de entrega y costos que exceden masivamente las estimaciones iniciales.

## Estudio de Casos de Falla

Los errores de software no deben entenderse como simples "bugs", sino como fallos de ingeniería con impactos económicos y humanos devastadores:

| Proyecto | Error Técnico | Impacto Derivado |
|----------|---------------|------------------|
|----------|---------------|------------------|

|                                    |                                                                                         |                                                                                          |
|------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>Ariane 5 (1996)</b>             | Conversión de un número real de 64 bits (coma flotante) a un entero de 16 bits.         | Explosión del cohete; pérdida de <b>500 millones de dólares</b> .                        |
| <b>Mars Climate Orbiter (1999)</b> | Inconsistencia de unidades: un equipo utilizó sistema inglés y otro el sistema métrico. | Pérdida de contacto con la sonda; <b>655 millones de dólares</b> .                       |
| <b>Toyota Prius (2010/14)</b>      | Fallo en el software de control de frenos (ABS) y sistema híbrido.                      | Llamada a revisión de 1.9 millones de vehículos; impacto de <b>3 billones (USD 3B)</b> . |

## El Pensamiento de Frederick Brooks: Esencia vs. Accidente

En su ensayo fundacional *"No Silver Bullet"*, Brooks distingue dos categorías de dificultades:

- **Dificultades Accidentales:** Barreras técnicas que acompañan la producción pero no son inherentes a la esencia del software (e.g., sintaxis compleja, tiempos de compilación). La tecnología moderna y la IA han mitigado estas barreras casi en su totalidad.
- **Dificultades Esenciales:** Propiedades irreducibles que impiden la existencia de una solución mágica o "bala de plata":
  1. **Complejidad:** El escalado del software aumenta los elementos interactuando de forma no lineal.
  2. **Conformidad:** La necesidad de adaptarse a interfaces humanas y sistemas arbitrarios sin principios unificadores.
  3. **Mutabilidad:** El software es "materia de pensamiento", infinitamente maleable y bajo presión de cambio continuo.
  4. **Invisibilidad:** El software carece de representación espacial o geométrica. Esta carencia de realidad física "priva a la mente de la abstracción visual", lo que dificulta enormemente la comunicación de la arquitectura conceptual.

## El Rol de la Inteligencia Artificial (IA)

La IA actúa como un catalizador para resolver lo accidental (generación de sintaxis), pero no logra vulnerar el **"cuello de botella humano"**. La parte más difícil de construir software no es

escribir el código, sino decidir el "Qué": la especificación, el diseño y la validación de la estructura conceptual. Si el ingeniero no comprende la complejidad esencial, el *prompt* será incompleto; la IA no puede deducir intenciones no declaradas ni "ver" la arquitectura si esta no ha sido visualizada previamente por el humano.

## 1. 2. LA INGENIERÍA DE SOFTWARE COMO DISCIPLINA FORMAL

La profesionalización del desarrollo es un imperativo estratégico. No basta con el dominio de la programación; se requiere la aplicación de estándares internacionales para garantizar previsibilidad y calidad. El marco de referencia global es el **SWEBOK** (*Software Engineering Body of Knowledge*).

### Definición Oficial (IEEE/SWEBOK)

"La ingeniería de software es la aplicación de un enfoque **sistemático, disciplinado y cuantificable** al desarrollo, operación y mantenimiento del software; es decir, la aplicación de los principios de la ingeniería al dominio del software".

### Clasificación Estructural de la Cátedra

Para el abordaje pedagógico en esta Facultad Regional, dividimos la disciplina en tres bloques:

1. **Disciplinas Técnicas:** Cubren el ciclo de vida desde la ingeniería de requerimientos hasta el despliegue.
2. **Disciplinas de Gestión:** Incluyen planificación, monitoreo y control. Es vital diferenciar el **seguimiento continuo** del estado diario frente al **control de hitos**, donde se verifica el cumplimiento de objetivos estratégicos.
3. **Disciplinas de Soporte (Protectoras):** Funcionan como un "paraguas" que garantiza la integridad. Aquí se ubican la Gestión de Configuración (SCM), el Aseguramiento de Calidad (SQA) y las Métricas.

Estas disciplinas se materializan mediante procesos definidos que luego son instanciados en proyectos específicos con restricciones temporales y de recursos.

## 1.3. MARCOS DE TRABAJO: PROCESOS VS. PROYECTOS

Es fundamental no confundir la teoría del flujo de trabajo con su ejecución:

- **Proceso:** Es el marco teórico o "libro de reglas" genérico (e.g., el Proceso Unificado o Scrum).
- **Proyecto:** Es la instanciación del proceso. Tiene un inicio y fin definidos, con personas, tiempo y presupuesto específicos.

La adaptación del proceso al proyecto (tailoring) es obligatoria y depende de factores como el **seniority** del equipo, la **tecnología**, la **críticidad** del sistema y el tamaño de la organización. Mientras que los **procesos definidos** buscan la predictibilidad manufacturera (poco aplicable al software), los **procesos empíricos** se basan en la inspección y adaptación constante para gestionar la complejidad.

## 2.GESTIÓN DE CONFIGURACIÓN (SCM)

### 2.1. FUNDAMENTOS DE SCM Y EL CONCEPTO DE INTEGRIDAD

La Gestión de Configuración de Software (SCM) es la disciplina que salvaguarda la **integridad** del producto. Según la definición técnica, es la "disciplina que aplica dirección y monitoreo administrativo y técnico" para: **identificar**, **controlar**, **registrar** (informar) y **verificar** cada elemento del sistema.

#### La Integridad y la Trazabilidad

La integridad se apoya en la capacidad de asegurar que el software cumple con los requerimientos y mantiene su consistencia interna. La herramienta primordial para esto es la **Matriz de Rastreabilidad**, que permite gestionar:

- **Trazabilidad Trasera (Pre-ERS):** Vincula los requerimientos con sus fuentes de origen (stakeholders, documentos de negocio).

- **Trazabilidad Delantera (Post-ERS):** Vincula los requerimientos con los artefactos de diseño, módulos de código y casos de prueba.

Debemos enfatizar que la SCM es fundamentalmente una cuestión de políticas y reglas. Las herramientas (Git, SVN) son meros facilitadores; sin una cultura de gestión, el repositorio más avanzado deviene en caos.

## 2.2. TAXONOMÍA Y CONCEPTOS CLAVE DE SCM

### Ítems de Configuración (IC / CI)

Un IC es cualquier artefacto que forma parte del producto o proyecto, susceptible de cambio y que debe ser compartido.

#### Tipos IC:

| Tipo de IC          | Duración                  | Ejemplos                                          |
|---------------------|---------------------------|---------------------------------------------------|
| <b>De Producto</b>  | Larga (Vida del software) | Código fuente, Manual de usuario, Arquitectura.   |
| <b>De Proyecto</b>  | Acotada (Gestión)         | Plan de Gestión (PGC), Cronograma, Riesgos.       |
| <b>De Iteración</b> | Corta (Ciclo de trabajo)  | Planes de iteración, Casos de prueba específicos. |

**Repositorio:** Almacén lógico de artefactos.

### Versión vs. Variante

- **Versión:** Forma de un artefacto en un instante dado de su evolución (e.g., v1.1).

- **Variante:** Instancia paralela para un contexto distinto (e.g., versión para Android vs. iOS) que coexisten en el tiempo.

## Normativa de la Cátedra: Reglas de Nombrado

Queda estrictamente prohibido incluir números de versión en el nombre físico de los archivos dentro del repositorio (e.g., NO usar Documento\_v1\_final.pdf). El repositorio gestiona la dimensión temporal y la historia; el nombre del archivo debe permanecer estable para no romper enlaces y referencias.

## Línea Base (Baseline)

Es un hito aprobado que sirve como cimiento para trabajos futuros. Representa un estado estable y verificado del sistema al cual se puede retornar ante cualquier contingencia.

## 2.3. LAS CUATRO ACTIVIDADES FUNDAMENTALES DE LA SCM

1. **Identificación:** Establecer esquemas de nomenclatura y estructuras de carpetas.
2. **Control de Cambios y el CCB:** Todo cambio debe formalizarse mediante una **Propuesta de Cambio (ECP)**. La evaluación recae en el **Comité de Control de Cambios (CCB)**, organismo multidisciplinario compuesto por representantes de Análisis, Diseño, Implementación y Testing.
3. **Auditorías de Configuración:**

| Auditoría              | Enfoque        | Objetivo                                                                               |
|------------------------|----------------|----------------------------------------------------------------------------------------|
| <b>Funcional (FCA)</b> | Requerimientos | Verifica que el producto construido cumple con la especificación funcional.            |
| <b>Física (PCA)</b>    | Inventario     | Asegura que los ICs definidos en la línea base realmente existen y están actualizados. |

1. **Informes de Estado (Accounting):** Registro del "Quién, Qué y Cuándo". Es la memoria histórica que permite la rendición de cuentas técnica.

## 2.4. EL PLAN DE GESTIÓN DE CONFIGURACIÓN (SCM PLAN)

El Plan de SCM es el **contrato operativo** del equipo. Debe definir:

- Roles y responsabilidades claras.
- Herramientas y flujo de trabajo de ramificación (Branching model).
- Criterios de creación de líneas base.
- Plantillas para ECPs y auditorías.

## 2.5. SCM EN ENTORNOS ÁGILES Y LEAN

En el paradigma moderno, la SCM evoluciona hacia el **Lean SCM**, cuyo objetivo principal es la eliminación del "desperdicio" (*waste*). Este desperdicio se manifiesta principalmente en **ramas de larga duración** y en el **infierno de la integración** (*integration hell*).

### Jerarquía de Builds (Berczuk)

Para garantizar la estabilidad en ciclos rápidos, se implementan tres niveles:

1. **Private System Build:** Construcción y prueba local en la estación del desarrollador.
2. **Integration Build:** Integración frecuente y automática en un ambiente compartido para detectar conflictos tempranamente.
3. **Release Build:** Versión candidata con máximos niveles de verificación para el despliegue final.

El flujo de **CI/CD** representa la maduración técnica de estas actividades, automatizando la verificación de integridad y permitiendo que la ingeniería de software mantenga su rigor a la velocidad que demanda la industria actual.

**Conclusión:** La Ingeniería de Software, soportada por una SCM robusta, es la única garantía de éxito profesional. El dominio de estas disciplinas protectoras distingue al ingeniero del mero programador, permitiendo gestionar la complejidad con integridad y previsibilidad técnica.



# 1. La Problemática de los Requerimientos y la Crisis de los Procesos Tradicionales

La ingeniería de software, definida magistralmente por Parnas como la "*construcción multipersonal de software multiversión*", enfrenta desde la conferencia de la NATO en 1968 una crisis de identidad técnica y estratégica. La fase de requerimientos constituye el punto crítico de falla; no por una incapacidad de codificación, sino por una deficiente alineación ontológica entre la necesidad del negocio y el producto construido. La historia nos provee evidencia empírica de este desajuste mediante catástrofes de alto impacto económico y humano: la destrucción del Mariner I (1962) por una fórmula mal programada (\$18.5 millones), el error sistémico en los frenos de Toyota (2010) que superó los \$3 billones en costos de mitigación, o los \$296.7 billones invertidos globalmente para subsanar la imprevisión del Y2K.

## Análisis de la Cita Fundacional de Frederick Brooks

En su ensayo '*No Silver Bullet*' (1987), Frederick Brooks establece una distinción fundamental para el liderazgo técnico:

- **La Esencia:** Es la construcción de estructuras conceptuales entrelazadas altamente complejas (algoritmos, flujos de datos, relaciones). Brooks sostiene que la parte más difícil de construir software es decidir *qué* decir, no cómo expresarlo.
- **El Accidente:** Son las dificultades de representación técnica (sintaxis de lenguajes, limitaciones de memoria, tiempos de compilación).

La historia del software es la eliminación sistemática de lo accidental mediante lenguajes de alto nivel e IDEs, pero la Esencia resiste la automatización porque es un acto de diseño conceptual puro.

## Las 4 Propiedades Irreducibles del Software Complejo

Para comprender la dificultad de los requerimientos, debemos analizar las características únicas del software según la cátedra:

1. **Complejidad:** Escalar el software aumenta los elements interactuando de forma no lineal. A diferencia de la manufactura, no hay piezas repetidas a nivel macro.

2. **Conformidad:** El software debe adaptarse a interfaces humanas arbitrarias y sistemas sin principios unificadores dictados por el entorno.
3. **Mutabilidad:** Es pura "*materia de pensamiento*", infinitamente maleable y constantemente sometida a la presión del cambio continuo.
4. **Invisibilidad:** El software carece de representación espacial o geométrica. No puede dibujarse en un plano, lo que priva a la mente de la abstracción visual directa.

## 2. Fundamentación de los Requerimientos Ágiles y Filosofía Lean

La agilidad no es rapidez; es la capacidad de respuesta al cambio mediante ciclos de retroalimentación cortos que mitigan las propiedades irreducibles mencionadas. Se fundamenta en el Manifiesto Ágil, impactando directamente en la colaboración diaria (Principio 4), la aceptación de requerimientos cambiantes (Principio 2) y la simplicidad como el arte de maximizar el trabajo no realizado (Principio 10).

### Los 3 Pilares y la Filosofía Lean

La gestión ágil se apoya en:

1. **Foco en el Valor de Negocio:** El software es conocimiento representado en niveles de abstracción; si no genera valor, es desperdicio.
2. **Just in Time (JIT):** Los detalles se postergan hasta el "último momento responsable" para alimentar el empirismo.
3. **Modelos en Equipos Autoorganizados:** El equipo técnico traduce la visión en un incremento funcional.

Bajo la óptica Lean, el objetivo es "crecer, no construir" (*grow, not build*), desarrollando un esqueleto mínimo funcional e incrementándolo orgánicamente para evitar el inventario de requerimientos obsoletos.

## 3. El Costo del Tradicional BRUF (Big Requirements Up Front)

Intentar definir el 100% del alcance al inicio es una falacia técnica con consecuencias económicas severas en el ROI. Según los datos analizados por la cátedra, el éxito de un proyecto está fuertemente correlacionado con el involucramiento del usuario (15.9%) y la existencia de

enunciados claros de requerimientos (9.6%). Por el contrario, la causa principal de fracaso son los requerimientos incompletos (13.1%) seguidos por la falta de participación del usuario (12.4%).

El enfoque BRUF ignora que el aprendizaje ocurre durante la construcción. La agilidad propone que el Product Backlog sea un ente vivo, un contenedor transitorio y evolutivo, descartando la idea de una especificación cerrada que solo genera funcionalidades nunca utilizadas y desperdicio procesal.

## 4. Captura, Elicitación y Persistencia (Tradicional vs. Ágil)

La documentación en agilidad no desaparece, se vuelve Documentación Lean. Mientras que el PUD (Proceso Unificado) concentra el peso documental en las fases de Inicio y Elaboración buscando una "estabilidad" teórica, la agilidad distribuye el relevamiento de forma continua.

- **ERS vs. Product Backlog:** La Especificación de Requerimientos de Software (ERS) tradicional es un contrato estático; el Backlog es un artefacto dinámico de gestión.
- **Ítems de Configuración (IC):** En el marco de la SCM (Administración de Configuración), los requerimientos, prototipos y casos de prueba son Ítems de Configuración sujetos a evolución.
- **Persistencia:** La verdadera persistencia del requerimiento reside en el código y en las pruebas automatizadas. El software funcionando es la única medida de progreso real.

## 5. Clasificación y Jerarquía de Requerimientos en Ágil

Separar el dominio del problema (el "qué") del dominio de la solución (el "cómo") es un imperativo para evitar la dominancia técnica sobre el negocio.

### La Pirámide de Requerimientos

- **Requerimientos de Negocio:** Objetivos estratégicos y visión.
- **Requerimientos de Usuario:** Representados por User Stories.
- **Requerimientos de Software:** Atributos de calidad (No Funcionales) y especificaciones técnicas.
- **Requerimientos de Implementación:** Infraestructura y estándares del SWEBOK.

Inversión de la Triple Restricción: A diferencia del modelo tradicional de alcance fijo, la agilidad fija el tiempo y el costo (iteraciones), permitiendo que el alcance sea la variable de ajuste para maximizar el valor entregado.

## 6. User Stories (Historias de Usuario) en Profundidad

Una User Story no es un documento; es una promesa de conversación futura. Se estructura bajo las 3 Cs de Ron Jeffries:

1. **Card (Tarjeta):** Soporte físico para la prioridad y el recordatorio del valor ("*Para [valor], como [rol], quiero [acción]*").
2. **Conversation (Conversación):** El núcleo del descubrimiento. La comunicación cara a cara reduce la ambigüedad inherente a la palabra escrita.
3. **Confirmation (Confirmación):** Los Criterios de Aceptación que validan el éxito.

## Vertical Slicing (Porciones Verticales)

A diferencia del enfoque del PUD, que a menudo busca estabilizar capas horizontales (ej. arquitectura de datos) antes de la funcionalidad, el Vertical Slicing exige que cada historia atraviese todas las capas técnicas. Esto permite validar valor funcional desde el primer incremento, evitando el riesgo de integrar silos tecnológicos al final del proyecto.

## 7. Calidad de las Historias de Usuario - Criterio INVEST

Para que una historia sea apta para el desarrollo, debe cumplir con el estándar INVEST (Mike Cohn):

- **Independent:** Evitar dependencias que bloqueen la prioridad.
- **Negotiable:** No es un contrato; es una invitación al diálogo.
- **Valuable:** Debe beneficiar al cliente, no ser solo una tarea técnica.
- **Estimable:** El equipo debe poder medir su complejidad y riesgo.
- **Small:** Debe caber holgadamente en una iteración.
- **Testable:** Debe ser validable objetivamente.

Gestión Binaria de Avance: En agilidad, el progreso es 0% o 100%. Una historia "casi terminada" es desperdicio procesal y no suma a la velocidad del equipo.

## 8. Diseño de Criterios y Pruebas de Aceptación

El diseño de criterios de aceptación requiere un análisis riguroso de casos reales. Tomemos como ejemplo el registro del 'Maestro Marcelo López': un diseño conceptual robusto debe prever el impacto de valores nulos o campos vacíos en el sistema. ¿Qué sucede si el apellido es compuesto? ¿Cómo reacciona el sistema ante entradas vacías?

El equipo debe diseñar:

- **Happy Path:** El flujo de éxito esperado.
- **Unhappy Paths:** Manejo de validaciones, límites, tipos de datos erróneos y sobrecargas.

## 9. Estimación Ágil y Planificación de la Capacidad

La estimación es un juicio humano sobre un esfuerzo intelectual, por lo tanto, es relativa.

- **Story Points (SP):** Unidad que amalgama complejidad, esfuerzo y riesgo.
- **Historia Canónica:** Punto de referencia conocido para comparaciones consistentes.
- **Planning Poker:** Técnica basada en el consenso para evitar el sesgo de anclaje y fomentar la discusión técnica.
- **Spikes:** Historias de investigación para reducir la incertidumbre técnica antes de comprometer una funcionalidad al Backlog.

## 10. Definiciones Adicionales de Gestión de Requerimientos en Scrum

La salud de un proyecto Scrum depende de definiciones claras que actúen como filtros de calidad:

1. **Definition of Ready (DoR):** Filtro de entrada. Asegura que la historia cumple con INVEST y tiene criterios de aceptación claros.
2. **Definition of Done (DoD):** Lista de control técnica (code review, unit tests, documentación) que garantiza que el incremento es potencialmente desplegable.
3. **Product Backlog Refinement (Grooming):** Actividad continua para desglosar y clarificar el trabajo futuro.

Finalmente, el ciclo de vida del requerimiento se cierra con la Auditoría Funcional de Configuración. Esta práctica asegura que el producto construido es consistente con los requerimientos aprobados y que la documentación técnica refleja fielmente la realidad del sistema, cumpliendo así con los estándares de calidad del IEEE y garantizando la entrega de software funcional de alto valor.